

Introdução à Linguagem Java

Universidade Estadual da Paraíba

Curso: Ciência da Computação

Componente curricular: Linguagem de Programação II

O que vamos aprender hoje?

- ① Linguagens **compiladas** x **interpretadas**
- ② Java: a linguagem **híbrida**
- ③ Nosso primeiro programa (*Hello World*) direto no terminal
- ④ Como ler e escrever dados em Java (Scanner)

- 1 Compilação x Interpretação
- 2 Java: a Linguagem Híbrida
- 3 Primeiro Programa em Java
- 4 Entrada e Saída com Scanner

- 1 Compilação x Interpretação
- 2 Java: a Linguagem Híbrida
- 3 Primeiro Programa em Java
- 4 Entrada e Saída com Scanner

O que acontece quando executamos um programa?

- Escrevemos código em uma linguagem que **nós** entendemos (Java, Python, C, ...);
- Mas o computador só entende **0 e 1** (linguagem de máquina);
- Alguém precisa traduzir um para o outro!

Existem **duas grandes estratégias** para fazer essa tradução.

Compilado x Interpretado

Compilado

Traduz tudo *antes* de rodar

Interpretado

Traduz *enquanto* roda, linha a linha

Vamos entender cada um com calma.

😊 Definição: Compilação

É o processo em que um programa chamado **compilador** lê *todo* o código-fonte de uma vez e gera um arquivo executável, já traduzido para linguagem de máquina.

Depois de compilado, o executável roda **diretamente** no sistema operacional – não precisa mais do código original nem do compilador.

Compilação: uma analogia

😊 Analogia

Imagine um tradutor que recebe um **livro inteiro** em inglês e traduz **o livro todo** para o português antes de te entregar. Você só recebe o resultado final – já pronto para ler do início ao fim.

O Processo de Compilação



Traduz tudo de uma vez, antes de o programa rodar

Exemplo: a linguagem C

☺ Exemplo: Compilando em C

No terminal, compilamos e depois executamos em dois passos:

```
gcc Programa.c -o Programa  
./Programa
```

O primeiro comando gera o executável; o segundo o executa.

Vantagens e Desvantagens: Compilado

Vantagens

- Executa muito rápido (já está em linguagem de máquina);
- O usuário final não precisa do código-fonte.

Desvantagens

- Precisa recompilar a cada alteração no código;
- O executável costuma funcionar só no sistema operacional para o qual foi compilado.

😊 Definição: Interpretação

É o processo em que um programa chamado **interpretador** lê o código-fonte **linha a linha** e já vai executando cada linha, sem gerar um executável separado.

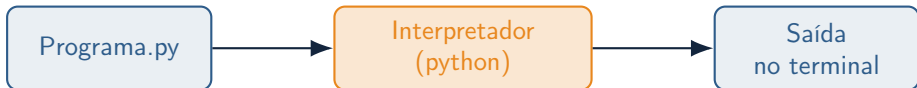
Para rodar de novo, o interpretador lê o código-fonte outra vez, começando pela primeira linha.

Interpretação: uma analogia

Analogia

Agora imagine um intérprete numa palestra: a pessor^a fala uma frase, o intérprete traduz **aquela frase** na hora, depois a próxima frase, e assim por diante – em tempo real, sem esperar a palestra terminar.

O Processo de Interpretação



Lê e executa uma linha de cada vez, sem gerar arquivo executável

Exemplo: a linguagem Python

☺ Exemplo: Rodando em Python

No terminal, um único comando já lê e executa o programa:

```
python Programa.py
```

Não existe um passo separado de compilação.

Vantagens e Desvantagens: Interpretado

Vantagens

- Não precisa compilar: é só rodar;
- Mais fácil de testar e depurar rapidamente.

Desvantagens

- Costuma ser mais lento que um programa compilado;
- Quem for executar precisa ter o interpretador instalado.

Comparando C x Python

	C (compilado)	Python (interpretado)
Precisa compilar?	Sim	Não
Gera executável?	Sim	Não
Velocidade de execução	Muito rápida	Mais lenta
Portabilidade do resultado	Depende do SO	Roda onde tiver o interpretador

♡ Checkpoint

- **Compilado:** traduz tudo antes, gera executável, roda sozinho depois;
- **Interpretado:** traduz e executa linha a linha, sempre precisa do interpretador.

Antes de seguirmos: essa distinção ficou clara? Guarde-a, porque o Java vai misturar as duas ideias!

Sumário

- 1 Compilação x Interpretação
- 2 Java: a Linguagem Híbrida**
- 3 Primeiro Programa em Java
- 4 Entrada e Saída com Scanner

Então, onde entra o Java?

E se eu disser que o Java **não é** totalmente compilado
nem totalmente interpretado?

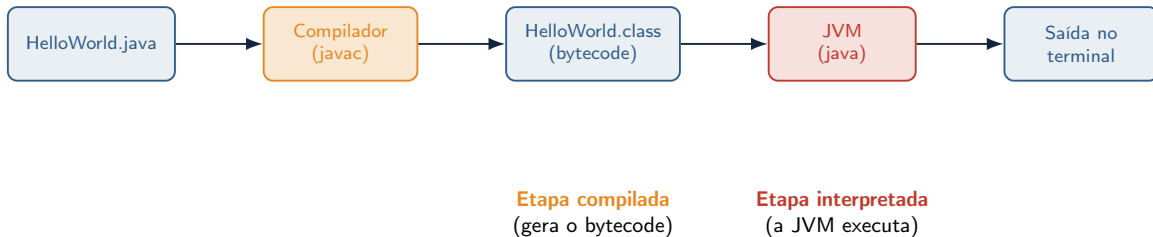
O Java usa um pouco de **cada um** dos dois processos.

😊 Definição: Linguagem híbrida

O Java primeiro é **compilado** para um código intermediário chamado **bytecode**. Depois, esse bytecode é **interpretado** (e otimizado) por uma máquina virtual, a **JVM**.

Ou seja: o Java sofre **dois** processos, um depois do outro.

O Processo Híbrido do Java



Passo a Passo do Processo Java

- 1 Escrevemos o código-fonte em um arquivo `.java`;
- 2 O comando `javac` **compila** esse arquivo e gera um arquivo `.class` (bytecode);
- 3 O comando `java` chama a **JVM**, que **interpreta** (executa) esse bytecode.

`javac` compila → `java` interpreta

O que é a JVM?

😊 Definição: JVM – Java Virtual Machine

É um programa que simula um "computador virtual" capaz de entender e executar byte-code Java. Existe uma JVM para cada sistema operacional (Windows, Linux, macOS).

É a JVM quem faz a ponte entre o bytecode (igual para todo mundo) e o computador real de cada pessoa.

"Escreva uma vez, rode em qualquer lugar"

- O arquivo `.class` (bytecode) é **o mesmo** em qualquer sistema operacional;
- Cada sistema operacional só precisa ter sua **própria JVM** instalada;
- Por isso um mesmo programa Java roda em Windows, Linux ou macOS sem ser recompilado.

Analogia

É como traduzir um livro para um **idioma universal** intermediário. Qualquer pessoa, em qualquer país, consegue ler esse idioma universal desde que tenha o seu próprio "leitor" (a JVM) para interpretá-lo em tempo real.

Espectro: Compilado $\leftrightarrow$ Interpretado

Totalmente Compilado

C, C++

Java

compila para bytecode,
a JVM interpreta

Totalmente Interpretado

Python, JavaScript

♡ Checkpoint

- Java **compila** o código-fonte (.java) em bytecode (.class);
- Depois, a **JVM interpreta** esse bytecode;
- Isso dá ao Java portabilidade (roda em qualquer SO) sem abrir mão de desempenho.

Sumário

- 1 Compilação x Interpretação
- 2 Java: a Linguagem Híbrida
- 3 Primeiro Programa em Java**
- 4 Entrada e Saída com Scanner

Vamos praticar!



Chegou a hora de escrever, compilar e executar
o nosso primeiro programa em Java.

Tudo isso direto no **terminal**, sem IDE – para vermos exatamente o que acontece por trás dos
panos.

Passo 1: criando o arquivo

Dica

O nome do arquivo **precisa** ser igual ao nome da classe pública que ele contém, com a extensão `.java`.

Como nossa classe vai se chamar HelloWorld, o arquivo deve se chamar:

`HelloWorld.java`

O Código Completo

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("Hello, World!");  
4     }  
5 }
```

Só isso! Vamos entender **cada pedacinho** dele.

Antes de quebrar o código...

Para entender o Hello World, precisamos primeiro de **3 ideias** que vão aparecer o tempo todo em Java:

- ❶ O que é uma **classe**?
- ❷ O que é um **método**?
- ❸ O que significa a palavra **public**?

O que é uma Classe?

😊 Definição: Classe

Em Java, **todo** código precisa morar dentro de uma classe. A classe é como uma **caixa organizadora**: ela guarda e dá nome ao que o programa sabe fazer.

Por enquanto, pense assim: sem uma classe, o Java nem sabe por onde começar a procurar o seu código.

Classe: uma analogia

😊 Analogia

Pense numa classe como uma **pasta** no computador. Sozinha, uma pasta vazia não faz nada – mas é dentro dela que guardamos os arquivos (no nosso caso, os métodos) que realmente fazem alguma coisa.

Mais pra frente no curso vamos ver que uma classe pode ser bem mais do que isso – mas isso já basta para começarmos.

O que é um Método?

😊 Definição: Método

É um **bloco de código com um nome**, que executa uma tarefa específica quando é chamado.

Todo método fica **dentro** de uma classe – é um dos "arquivos" guardados na nossa "pasta".

Método: uma analogia

Analogia

Pense num método como um **botão**: quando alguém aperta o botão (chama o método), uma sequência de ações acontece, na ordem escrita ali dentro. O nome do botão diz o que ele faz.

O que significa `public`?

😊 Definição: `public`

É uma palavra que controla **quem pode acessar** algo – uma classe, um método, etc. `public` diz: "qualquer parte do programa pode usar isto".

😊 Analogia

Pense em `public` como a **porta principal** de um prédio: qualquer pessoa pode entrar por ela. Mais pra frente veremos palavras como `private`, que funcionam como uma porta trancada, só acessível de dentro.

♥ Checkpoint

- **Classe:** a "caixa organizadora" que guarda o código;
- **Método:** um "botão" com um nome, que executa uma tarefa;
- **public:** "porta aberta" – qualquer parte do programa pode acessar.

Com essas 3 ideias, já conseguimos ler o Hello World de verdade!

Quebrando o código em pedaços

Vamos analisar linha por linha, palavra por palavra:

- ① `public class HelloWorld`
- ② `public static void main(String[] args)`
- ③ `String[] args`
- ④ As chaves `{ }`
- ⑤ `System.out.println(...)`
- ⑥ O ponto e vírgula `;`

Pedaço 1: a classe

```
1 public class HelloWorld {
```

- `public`: lembra da porta aberta? Qualquer parte do programa pode usar esta classe;
- `class`: estamos declarando uma **classe** – a nossa "caixa organizadora";
- `HelloWorld`: o **nome** da classe – precisa bater com o nome do arquivo.

Pedaço 2: public static

```
1 public static void main(String[] args) {
```

- `public`: de novo a porta aberta – qualquer parte do programa pode chamar este método;
- `static`: por enquanto, pense nisso como "obrigatório para o `main` funcionar sem precisarmos criar nada antes". Vamos entender esse detalhe com calma quando estudarmos objetos, mais à frente no curso.

Pedacço 2: void main

```
1 public static void main(String[] args) {
```

- void: esse método **não devolve** nenhum valor;
- main: é o nome especial que a JVM procura para saber **por onde começar** a executar o programa.

😊 Dica

Todo programa Java precisa de um método `main` para poder ser executado diretamente.

Pedaço 3: String[] args

```
1 public static void main(String[] args) {
```

- String[]: um **vetor (array)** de textos;
- args: o nome dessa variável – guarda argumentos digitados no terminal na hora de rodar o programa.

Por enquanto não vamos usar o args, mas ele sempre precisa estar aí.

Pedaço 4: as chaves { }

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3  
4     }  
5 }
```

- Toda chave { que **abre** um bloco precisa de uma } que o **fecha**;
- A classe é um bloco; o método main é outro bloco **dentro** dela.

Pedaço 5: System.out.println

```
System.out.println("Hello, World!");
```

- System: uma classe pronta do Java que representa o "sistema";
- out: representa a **saída padrão** – normalmente, o terminal;
- println: método que **imprime** o texto e pula para a próxima linha.

print x println

```
1 System.out.print("Ola");  
2 System.out.println(" Mundo");  
3 System.out.println("Fim");
```

- print: imprime e **continua na mesma linha**;
- println: imprime e **pula** para a linha seguinte.

Saída no terminal:

```
Ola Mundo  
Fim
```

Pedaco 6: o ponto e vírgula

```
System.out.println("Hello, World!");
```

- Em Java, cada **instrução** termina com um ;
- É como o ponto final de uma frase – avisa "essa instrução acabou aqui".

☹ Erro Comum

Esquecer o ; é um dos erros mais comuns de quem está começando em Java!

O Código Completo, de Novo

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("Hello, World!");  
4     }  
5 }
```

Agora cada peça já faz sentido!

Hora do Terminal: compilando

Com o arquivo HelloWorld.java salvo, abrimos o terminal na mesma pasta e digitamos:

```
javac HelloWorld.java
```

- javac é o **compilador** do Java;
- Se não aparecer nenhum erro, nada é impresso na tela – é sinal de que deu certo!

O que aconteceu? O arquivo .class

Depois do `javac`, um novo arquivo aparece na pasta:

`HelloWorld.class`

- Esse é o **bytecode** – a tradução do nosso código para a "linguagem" que a JVM entende;
- Não é texto legível como o `.java`; é um formato binário, feito para a JVM.

Hora do Terminal: executando

Agora rodamos o programa chamando a JVM:

```
java HelloWorld
```

- Repare: **sem** a extensão `.class`;
- A JVM procura o arquivo `HelloWorld.class` e interpreta o bytecode dele;
- Resultado no terminal:

```
Hello, World!
```

O Fluxo Completo



Erro Comum

Tentar executar `java HelloWorld.class` (com a extensão) em vez de `java HelloWorld`. O comando `java` já espera o **nome da classe**, sem extensão nenhuma.

Erro Comum

Salvar o arquivo com um nome diferente do nome da classe pública – por exemplo, salvar `Programa.java` contendo `public class HelloWorld`. O `javac` não compila!

Dica

Mesmo depois de começar a usar uma IDE, entender o que `javac` e `java` fazem por baixo dos panos ajuda **muito** a resolver erros – porque é exatamente isso que a IDE está fazendo por você, escondido em um botão "Run".

♡ Checkpoint

- Arquivo `.java` → `javac` → arquivo `.class` (bytecode);
- `java NomeDaClasse` → a JVM interpreta o bytecode e executa;
- Nome do arquivo = nome da classe pública, sempre.

- 1 Compilação x Interpretação
- 2 Java: a Linguagem Híbrida
- 3 Primeiro Programa em Java
- 4 Entrada e Saída com Scanner**

Por que precisamos de entrada de dados?

- Até agora, nosso programa sempre imprime a **mesma** coisa;
- Programas úteis normalmente reagem ao que o **usuário** digita;
- Precisamos de uma forma de **ler** dados do teclado.

Em Java, quem faz isso é a classe **Scanner**. 😊

Apresentando o Scanner

😊 Definição: Scanner

É uma classe pronta do Java, usada para **ler** dados digitados pelo usuário no terminal (ou vindos de outras fontes de entrada).

Import necessário

```
1 import java.util.Scanner;
```

- O Java organiza suas classes prontas em **pacotes**;
- Scanner mora no pacote `java.util`;
- Por isso, para usá-la, precisamos importá-la **antes** da nossa classe.

Criando o objeto Scanner

```
Scanner leitor = new Scanner(System.in);
```

- Scanner: o **tipo** da variável;
- leitor: o **nome** que demos a essa variável;
- new Scanner(...): criamos um novo objeto Scanner;
- System.in: dizemos que a fonte de leitura é a **entrada padrão** – normalmente, o teclado.

Método	O que lê
<code>nextInt()</code>	um número inteiro
<code>nextDouble()</code>	um número decimal
<code>next()</code>	uma palavra (sem espaço)
<code>nextLine()</code>	uma linha inteira (com espaços)

☺ Exemplo: Cadastro simples

```
1 import java.util.Scanner;
2
3 public class Cadastro {
4     public static void main(String[] args) {
5         Scanner leitor = new Scanner(System.in);
6         System.out.print("Digite seu nome: ");
7         String nome = leitor.nextLine();
8         System.out.print("Digite sua idade: ");
9         int idade = leitor.nextInt();
10        System.out.println("Ola, " + nome + "!");
11        System.out.println("Voce tem " + idade + " anos.");
12        leitor.close();
13    }
14 }
```

Explicando o exemplo

- `System.out.print` pergunta, **sem** pular linha, para o cursor ficar ao lado da resposta;
- `leitor.nextLine()` guarda o que foi digitado em `nome` (um `String`);
- `leitor.nextInt()` guarda o número digitado em `idade` (um `int`);
- O `+` entre textos e variáveis **concatena** (junta) tudo em uma única mensagem.

Erro Comum: nextInt() seguido de nextLine()

```
1 int idade = leitor.nextInt();  
2 String nome = leitor.nextLine(); // vem vazio!
```

☹ Erro Comum

O `nextInt()` lê o número, mas **deixa** na fila o "Enter" (quebra de linha) que veio logo depois. O `nextLine()` seguinte lê justamente essa quebra de linha sobrando – e retorna uma string vazia.

Dica: como resolver

😊 Dica

Depois de um `nextInt()` (ou `nextDouble()`), antes de usar `nextLine()`, consuma o "Enter" sobrando com um `nextLine()` extra:

```
1 int idade = leitor.nextInt();  
2 leitor.nextLine(); // descarta o Enter pendente  
3 String nome = leitor.nextLine(); // agora funciona
```

Fechando o Scanner

```
1 leitor.close();
```

- É uma boa prática fechar o Scanner quando terminamos de usá-lo;
- Libera recursos que o programa estava usando para ler a entrada.

Dica

Feche o Scanner só no **fim** do programa – depois de fechado, ele não pode mais ler nada.

♡ Checkpoint

- `import java.util.Scanner; → Scanner leitor = new Scanner(System.in);`
- `nextInt`, `nextDouble`, `next`, `nextLine` leem tipos diferentes;
- Cuidado ao misturar `nextInt()` com `nextLine()`;
- Feche o Scanner no final com `close()`.

Antes de programar, precisamos preparar o computador!

- ❶ Instalar o **Java Development Kit (JDK)**;
- ❷ Verificar se o Java foi instalado corretamente;
- ❸ Instalar a **Eclipse IDE**;
- ❹ Criar nosso primeiro projeto Java.

Dica

Para a disciplina, utilizaremos o **JDK 25**, versão LTS (*Long-Term Support*) mais recente do Java.

Instalando o Java – JDK 25 LTS

1. Acesse o site oficial da Oracle

oracle.com/java/technologies/downloads

- 1 Procure por **Java SE Development Kit 25**;
- 2 Escolha o seu sistema operacional;
- 3 No Windows, escolha **x64 Installer**;
- 4 Execute o instalador e siga os passos normalmente.

[Download direto – Windows x64](#)

Verificando a Instalação do Java

Depois da instalação, abra o **Prompt de Comando** ou **PowerShell** e execute:

```
java --version
```

Em seguida:

```
javac --version
```

♥ Checkpoint

Se os dois comandos exibirem uma versão iniciada por **25**, o Java e o compilador foram instalados corretamente.

☹ Erro Comum

Se aparecer “java não é reconhecido” ou mensagem semelhante, o Java não foi encontrado no **PATH** do sistema.

Baixando o Eclipse Installer

① Acesse o site oficial:

eclipse.org/downloads

② Clique no botão **Download**;

③ No Windows, escolha:

Download x86_64

Dica

A opção **x86_64** é indicada para a maioria dos computadores com processadores Intel ou AMD.

Instalando o Eclipse – Passo 2

Baixando o instalador

Após selecionar **Download x86_64**, o Eclipse abrirá uma página para escolher o servidor de download.

- 1 O site selecionará automaticamente um servidor próximo;
- 2 Clique novamente no botão:

Download

- 3 Aguarde o download do arquivo:

`eclipse-inst-jre-win64.exe`

Dica

Não é necessário alterar o servidor sugerido pelo site. Basta clicar em **Download**.

Escolhendo o Eclipse para Java

Abra o arquivo que foi baixado e aguarde o **Eclipse Installer** iniciar.

Na lista apresentada, escolha:

Eclipse IDE for Java Developers

Finalizando a instalação

Após selecionar **Eclipse IDE for Java Developers**:

- 1 Confira a pasta onde o Eclipse será instalado;
- 2 Mantenha as opções padrão;
- 3 Clique em **INSTALL**;
- 4 Aceite os termos da licença, caso solicitado;
- 5 Aguarde a conclusão da instalação;
- 6 Clique em **LAUNCH** para abrir o Eclipse.

♥ Checkpoint

Ao abrir o Eclipse pela primeira vez, será solicitado um **Workspace**. Ele será a pasta onde seus projetos serão armazenados.

- ① Linguagens **compiladas** traduzem tudo antes; **interpretadas** traduzem linha a linha;
- ② **Java** faz as duas coisas: `javac` compila para bytecode (`.class`), a **JVM** interpreta esse bytecode;
- ③ Um programa Java sempre precisa de um método `main`, dentro de uma classe pública com o mesmo nome do arquivo;
- ④ O Scanner lê dados digitados pelo usuário no terminal.